



Control Panel

Select Model Engine ?

gemini-2.5-pro ▼

System: Vertex AI (ADC Enabled)

IBM Legacy Modernization Agent
v2.0

Legacy Scribe

1a. Upgrading large-scale legacy systems into intelligent, scalable, future-ready platforms.

Legacy Asset Input

Asset Type

COBOL Source ▼

Paste Code / Logs here:

[SCRIBE]: Deconstructing legacy syntax and state machine...

Modernization Strategy Generated

System Analysis

Logic Summary: The code checks if an account's balance is below a threshold (500). If it is, a fixed service charge (10.00) is deducted from the balance.

Identified Security Vulnerabilities

- Hardcoded business logic (threshold of 500, fee of 10.00) makes the system rigid and requires code

```
IF BALANCE < 500  
MOVE 10.00 TO SERVICE-CHARGE  
SUBTRACT SERVICE-CHARGE  
FROM BALANCE  
END-IF
```

Initiate Modernization Pipeline

changes for updates, increasing the risk of introducing errors during maintenance.

- Lack of input validation for the 'BALANCE' variable could lead to computational errors or system failures (e.g., a COBOL S0C7 abend) if non-numeric data is processed.
- The snippet lacks auditing or logging. It's impossible to track when and why fees are applied, which can be a financial compliance and customer dispute risk.

Target Architecture: Externalize this business rule from the application code into a dedicated Business Rules Engine (BRE) or a configuration service/database. This logic should be exposed as a stateless microservice (e.g., 'fee-calculation-service') that accepts an account state and returns the calculated fee. This allows for dynamic updates to rule parameters (like the fee amount and balance threshold) without redeploying the core application, improving agility and reducing risk.

Refactored Implementation (Python/FastAPI)

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

# These values should be loaded from c
# instead of being hardcoded.
LOW_BALANCE_THRESHOLD = 500.00
SERVICE_CHARGE = 10.00

class Account(BaseModel):
    account_id: str
    balance: float

class FeeCalculationResponse(BaseModel):
    original_balance: float
    new_balance: float
    service_charge_applied: float

@app.post("/fees/apply-low-balance-charge")
async def apply_service_charge(account: Account) >
    """
    Applies a service charge if the ac
    This is a stateless endpoint that
    """
    charge_applied = 0.0
    if account.balance < LOW_BALANCE_THRESHOLD:
```

```
        charge_applied = SERVICE_CHARGE  
  
        new_balance = account.balance - charge_applied  
  
    return FeeCalculationResponse(  
        original_balance=account.balance,  
        new_balance=new_balance,  
        service_charge_applied=charge_applied,  
    )
```

[Download Modernization Report](#)